

ОТЧЕТ
по лабораторной работе № 7
по дисциплине «Программирование на Python»
Тема: Логирование и обработка ошибок в Python

Выполнил: студент гр. Р3121
Баранова Елизавета Павловна

Преподаватель: Жуков Николай
Николаевич

Цели работы:

- освоить принципы разработки декораторов с параметрами;
- научиться разделять ответственность функций (бизнес-логика) и декораторов (сквозная логика);
- научиться обрабатывать исключения, возникающие при работе с внешними API;
- освоить логирование в разные типы потоков (sys.stdout, io.StringIO, logging);
- научиться тестировать функцию и поведение логирования.

Код декоратора с параметрами

```
import sys
import functools
import logging
import requests.exceptions

def logger(func=None, *, handle=sys.stdout):
    """
    Декоратор `logger` для логирования вызовов функций.

    Аргументы:
        func (callable, optional): Функция для декорирования. Если не передан,
        возвращается декоратор с настройками.
        handle (sys.stdout или logging.Logger, опционально): Поток или логгер
        для вывода сообщений. По умолчанию sys.stdout.
    """

    if func is None:
        return lambda real_func: logger(real_func, handle=handle)

    @functools.wraps(func)
    def inner(*args, **kwargs):
        kwargs.pop('handle', None)

        is_logger = isinstance(handle, logging.Logger)

        try:
            msg_start = f"INFO: Start {func.__name__} with args={args},\nkwargs={kwargs}\n"
            if is_logger:
                handle.info(msg_start)
            else:
                handle.write(msg_start)

            result = func(*args, **kwargs)

            msg_end = f"INFO: Finished {func.__name__} with result={result}\n"
            if is_logger:
                handle.info(msg_end)
```

```

        else:
            handle.write(msg_end)

        return result
    except requests.exceptions.RequestException as e:
        msg_err = f"ERROR: Ошибка при запросе к API: {e}\n"
        if is_logger:
            handle.error(msg_err)
        else:
            handle.write(msg_err)
        raise requests.exceptions.RequestException("Упали с исключением")
from e

return inner

```

Исходный код get_currencies

```

import requests
from decorators import logger
import sys
@logger(handle=sys.stdout)
def get_currencies(currency_codes: list, url: str = "https://www.cbr-xml-
daily.ru/daily_json.js") -> dict:
    """
    Получает курсы валют с API Центробанка России.
    """
    try:
        response = requests.get(url)
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        raise requests.exceptions.RequestException("API недоступен") from e

    try:
        data = response.json()
    except ValueError as e:
        raise ValueError("Некорректный JSON") from e

    if "Valute" not in data:
        raise KeyError("Ключ 'Valute' отсутствует в данных")

    currencies = {}
    for code in currency_codes:
        if code not in data["Valute"]:
            raise KeyError(f"Код валюты '{code}' не найден")
        value = data["Valute"][code]["Value"]
        if not isinstance(value, (int, float)):
            raise TypeError(f"Курс валюты '{code}' имеет неверный тип")
        currencies[code] = value

    return currencies

if __name__ == "__main__":
    currency_list = ['USD', 'EUR', 'GBP', 'NNZ']
    try:
        currency_data = get_currencies(currency_list)
        print(currency_data)
    except Exception as e:
        print(f"Произошла ошибка: {e}")

```

Демонстрационный пример (квадратное уравнение)

```
import sys
import logging
from decorators import logger

demo_logger = logging.getLogger("quadratic_demo")
demo_logger.setLevel(logging.DEBUG)
demo_logger.addHandler(logging.StreamHandler(sys.stdout))

@logger(handle=demo_logger)
def solve_quadratic(a, b, c):
    """
    Функция `solve_quadratic` решает квадратные уравнения вида  $ax^2 + bx + c = 0$ .

    Аргументы:
        a (int|float): коэффициент при  $x^2$ 
        b (int|float): коэффициент при  $x$ 
        c (int|float): свободный член
    """

    if not all(isinstance(x, (int, float)) for x in (a, b, c)):
        raise ValueError("Коэффициенты должны быть числами")

    if a == 0 and b == 0:
        raise ValueError("Уравнение не имеет смысла")

    if a == 0:
        return -c / b

    d = b ** 2 - 4 * a * c
    if d < 0:
        return f"Нет действительных корней (D={d})"

    if d == 0:
        return -b / (2 * a)

    x1 = (-b + d ** 0.5) / (2 * a)
    x2 = (-b - d ** 0.5) / (2 * a)
    return x1, x2
```

Скриншоты логов для примера с квадратным уравнением

```
INFO: Start solve_quadratic with args=('abc', 2, 1), kwargs={}

INFO: Start solve_quadratic with args=(1, 0, 5), kwargs={}

INFO: Finished solve_quadratic with result=Нет действительных корней (D=-20)

INFO: Start solve_quadratic with args=(1, 2, 1), kwargs={}

INFO: Finished solve_quadratic with result=-1.0

INFO: Start solve_quadratic with args=(1, -3, 2), kwargs={}

INFO: Finished solve_quadratic with result=(2.0, 1.0)
```

Скриншоты логов

```
currency.log x
1 INFO: Start get_currencies_file_logged with args=(['USD', 'EUR'],), kwargs={}
2
3 INFO: Finished get_currencies_file_logged with result={'USD': 76.0937, 'EUR': 88.7028}
4
5
```

Тесты

```
import unittest
import io
import requests

from get_currency import get_currencies
from decorators import logger
from solve_quadratic import solve_quadratic

class TestGetCurrencies(unittest.TestCase):
    """
    Проверяет функцию `get_currencies`.
    """

    def setUp(self):
        self.clean_get = get_currencies.__wrapped__

    def test_valid_currency(self):
        """Проверка корректного возврата существующих валют."""
        result = self.clean_get(['USD', 'EUR'])
        self.assertIn("USD", result)
        self.assertIn("EUR", result)

    def test_nonexistent_currency(self):
```

```

        """Проверка исключения KeyError для несуществующей валюты."""
        with self.assertRaises(KeyError):
            self.clean_get(['XYZ'])

    def test_connection_error(self):
        """Проверка RequestException при ошибке подключения."""
        with self.assertRaises(requests.exceptions.RequestException):
            self.clean_get(['USD'], url="https://invalid-url")

    def test_invalid_json(self):
        """Проверка ValueError при некорректном JSON."""
        with self.assertRaises(ValueError):
            self.clean_get(['USD'], url="https://example.com")

class TestDecoratorLogging(unittest.TestCase):
    """
    Проверяет декоратор logger.
    """

    def setUp(self):
        self.stream = io.StringIO()

        @logger(handle=self.stream)
        def wrapped_get(codes, url=None):
            return get_currencies.__wrapped__(codes, url)

        self.wrapped = wrapped_get

    def test_logging_connection_error(self):
        """Проверка логирования при ошибке подключения."""
        with self.assertRaises(requests.exceptions.RequestException):
            self.wrapped(['USD'], url="https://invalid-url")

        logs = self.stream.getvalue()
        self.assertIn("ERROR", logs)
        self.assertIn("Ошибка при запросе к API", logs)

    def test_logging_nonexistent_currency(self):
        """Проверка логирования при запросе несуществующей валюты."""
        with self.assertRaises(requests.exceptions.RequestException):
            self.wrapped(['XYZ'])

class TestSolveQuadratic(unittest.TestCase):
    """
    Проверяет функцию solve_quadratic.
    """

    def test_two_roots(self):
        """Проверка функции при двух корнях."""
        res = solve_quadratic(1, -3, 2)
        self.assertEqual(res, [1.0, 2.0])

    def test_one_root(self):
        """Проверка функции при одном корне."""
        res = solve_quadratic(1, 2, 1)
        self.assertEqual(res, [-1.0])

    def test_no_real(self):
        """Проверка функции при отсутствии действительных корней."""
        res = solve_quadratic(1, 0, 5)
        self.assertIn("Нет действительных корней", res)

```

```
def test_invalid(self):
    """Проверка исключения при некорректных аргументах."""
    with self.assertRaises(ValueError):
        solve_quadratic("abc", 2, 1)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Результаты тестов

```
/usr/local/bin/python3.12 /Applications/PyCharm CE.app/Contents/plugins/python-ce/
Testing started at 12:09 ...
Launching unittests with arguments python -m unittest /Users/маcbook/Вуз/Питон/Pyt

INFO: Start solve_quadratic with args=('abc', 2, 1), kwargs={}

INFO: Start solve_quadratic with args=(1, 0, 5), kwargs={}

INFO: Finished solve_quadratic with result=Нет действительных корней (D=-20)

INFO: Start solve_quadratic with args=(1, 2, 1), kwargs={}

INFO: Finished solve_quadratic with result=-1.0

INFO: Start solve_quadratic with args=(1, -3, 2), kwargs={}

INFO: Finished solve_quadratic with result=(2.0, 1.0)

Ran 10 tests in 1.441s

OK

Process finished with exit code 0
```